

AI Trip Planner

Memory System & Evaluation Framework

Deep Dive Documentation

Generated: 24 May 2026, 23:46
Author: Aravind G | Project: AI Trip Planner (Multi-Agent LangGraph)

Table of Contents

- Memory System Overview
- SQLite — Structured Storage (memory/session_store.py)
- FAISS Vector Store — Semantic Search (memory/vector_store.py)
- Memory Agent — Retrieval & Update (agents/memory_agent.py)
- How Memory Flows Through the Workflow
- Evaluation Framework Overview
- RAGAS Evaluation — 4 Metrics (utils/evaluator.py)
- DeepEval Evaluation — 5 Metrics (utils/evaluator.py)
- Final Grade Calculation
- Where Evaluation is Triggered (agents/pdf_generator_agent.py)
- Where Results are Displayed (app.py)
- End-to-End Order of Execution

1. Memory System Overview

The AI Trip Planner uses a **dual-store memory architecture**. Two completely different storage technologies work together to give the system both exact record-keeping and intelligent semantic recall. Neither store alone is sufficient — they complement each other.

Store	Technology	File	Best For
SQLite	Relational DB	data/trips.db	Exact lookups, full trip history, structured queries
FAISS	Vector Index	data/vector_store/{user_id}/	Semantic search, preference patterns, similarity

Together they answer two kinds of questions: SQLite answers 'What was my Goa trip budget?' while FAISS answers 'What kind of traveller am I?'

2. SQLite — Structured Storage

File: memory/session_store.py | Class: TripDatabase

Purpose

SQLite is a lightweight file-based relational database stored at **data/trips.db**. It acts as the permanent, structured record of every trip a user has planned. Unlike FAISS, SQLite stores the *complete* trip data — every field, every preference, the full generated plan JSON, and the PDF path.

Tables

Table	Columns	Purpose
trips	id, user_id, destination, source, travel_dates, budget, travel_preferences, trip_plan, json_plan, pdf_path	One-to-one record of trip, plan, json plan, pdf path.
user_profiles	user_id, name, email, preferences_json, updated_at	Stores last known preferences: preferred hotel, travel type, last c
sqlite_sequence	name, seq	Internal SQLite auto-increment tracker. Not user data.

Key Methods

Method	What it does
save_trip(user_id, trip_data)	Inserts a full trip record after workflow completion
get_user_trips(user_id, limit)	Returns last N trips for a user, ordered by date
save_user_profile(user_id, ...)	Upserts user profile (INSERT OR REPLACE)
get_user_profile(user_id)	Fetches user's last known preferences

When Data is Written

SQLite is written during the **memory_update_node** step — the second-to-last step in the workflow, after the review agent has approved the plan. Both tables are written in the same call.

How to Query (Python)

```
import sqlite3
from config import DB_PATH
with sqlite3.connect(DB_PATH) as conn:
    rows = conn.execute("SELECT * FROM trips WHERE user_id=?", ("arav",)).fetchall()
    print(rows)
```

Limitation

- *SQLite cannot do fuzzy or semantic search. You cannot ask it 'find trips similar to a beach holiday'. That is what FAISS is for.*

3. FAISS Vector Store — Semantic Search

File: memory/vector_store.py | Class: VectorMemory

What is FAISS?

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search over dense vectors. Instead of storing raw text, it converts text into a list of numbers (called an **embedding**) that captures the *meaning* of the text. Two sentences with similar meaning will have embeddings that are numerically close to each other.

Embeddings Used

The system uses **OpenAIEmbeddings** (text-embedding-ada-002) when an OpenAI API key is present in .env. Each trip summary is converted to a 1536-dimensional vector. If no OpenAI key is found, it falls back to FakeEmbeddings (random numbers) which disables real semantic search.

Condition	Embedding Used	Semantic Quality
OPENAI_API_KEY set in .env	OpenAIEmbeddings (real)	Full semantic similarity
No OPENAI_API_KEY	FakeEmbeddings (random)	No semantic meaning — random results

Storage Location

FAISS index is saved to disk at: `data/vector_store/{user_id}/faiss_index/`

Raw documents are also saved as JSON at: `data/vector_store/{user_id}/documents.json`

What Gets Stored

Each time a trip completes, the following text summary is embedded and stored:

```
"Trip to Goa from Bangalore for 5 days.  
Travel type: couple. Budget: 30000.  
Hotel preference: mid-range.  
Interests: beach, nightlife."
```

Along with metadata: destination, budget, hotel_preference, travel_type.

Key Methods

Method	What it does
add_memory(content, metadata)	Embeds text → adds to FAISS index → saves index to disk
search(query, k=3)	Embeds query → finds k most similar documents in FAISS
get_user_preferences()	Loops all stored docs → extracts avg_budget, preferred_hotel_type, preferred_travel_type, pas

How Semantic Search Works (Step by Step)

- User completes Trip 1 (Goa, couple, beach). Memory saved as vector V1.
- User starts Trip 2 (Andaman, couple). Query 'beach holiday couple' is embedded as vector Q.
- FAISS computes cosine distance between Q and all stored vectors.
- V1 (Goa beach trip) scores high similarity → returned as relevant past context.
- Agents receive this context and personalise the new plan accordingly.

How get_user_preferences() Works

This function does NOT use FAISS search. It simply loops through all stored document metadata and aggregates values:

- Collects all destination values → past_destinations list
- Averages all budget values → avg_budget

- Takes the last saved hotel_preference → preferred_hotel_type
- Takes the last saved travel_type → preferred_travel_type

■ *Bias towards most recent trip: since it uses the last saved value for hotel and travel type, the most recent trip always overrides older ones for these preferences.*

4. Memory Agent — Retrieval and Update

File: agents/memory_agent.py | Functions: memory_retrieval_node, memory_update_node

memory_retrieval_node (Step 2 in workflow)

Runs after user_input_node. Fetches data from both stores and combines them into a single user_preferences_memory dict that is passed through state to all subsequent agents.

```
vector_prefs = VectorMemory(user_id).get_user_preferences()
past_trips   = TripDatabase().get_user_trips(user_id, limit=3)
db_profile   = TripDatabase().get_user_profile(user_id)

# merge: vector prefs + db profile preferences
prefs = {**vector_prefs}
prefs.update(db_profile['preferences']) # db overrides vector
prefs['trip_count'] = len(past_trips)
```

Output stored in state as: user_preferences_memory

memory_update_node (Step 8 in workflow)

Runs after review approval, before PDF generation. Saves the current trip to both stores.

Store	What is saved
FAISS (VectorMemory)	Text summary + metadata (destination, budget, hotel_preference, travel_type)
SQLite trips table	Full trip record including plan_json and pdf_path
SQLite user_profiles table	Updated preferred_hotel, preferred_travel_type, last_destination

First-Time User Behaviour

For a brand new user, both stores return empty data. The retrieval node still runs and sets user_preferences_memory = {'trip_count': 0}. All agents fall back to using only what the user typed in the form. Memory becomes useful from the second trip onwards.

5. How Memory Flows Through the Workflow

Below shows which agents read from user_preferences_memory and what they do with it:

Agent	File	What memory is used for
hotel_agent	agents/hotel_agent.py	If user left hotel_preference as default 'mid-range', overrides with preferred_hotel
budget_agent	agents/budget_agent.py	If current trip budget is more than 20% below avg_budget, adds a warning sugg
itinerary_agent	agents/itinerary_agent.py	Passes past_destinations, trip_count, preferred_travel_type into LLM prompt to

■ *user_input_agent cannot use memory because it runs BEFORE memory_retrieval_node in the workflow order.*

6. Evaluation Framework Overview

After all agents complete and just before the PDF is generated, the system runs a **quality evaluation** of the generated trip plan. Two evaluation frameworks are implemented: RAGAS and DeepEval. Both are heuristic implementations — they mirror the interface of the real frameworks but use fast mathematical rules instead of expensive LLM API calls.

Framework	What it evaluates	Metrics	File
RAGAS	Quality of retrieval and grounding	4 metrics	utils/evaluator.py
DeepEval	Quality of generated output	5 metrics	utils/evaluator.py

The real RAGAS and DeepEval libraries (pip packages) are commented out in requirements.txt because they require LLM calls for every metric, making them slow and expensive. These heuristic versions run instantly with zero API cost.

7. RAGAS Evaluation — 4 Metrics

File: utils/evaluator.py | Class: RAGASEvaluator

RAGAS (Retrieval Augmented Generation Assessment) originally measures how well an LLM uses retrieved documents to generate answers. In this project it measures how well the trip plan uses the data gathered by the weather, hotel, transport, and places agents.

Metric 1 — Faithfulness

Question: Is the itinerary grounded in the actual data retrieved by the agents?

Formula:

```
score = 0.50 (base)
+ 0.20 if the recommended hotel name appears in the itinerary's accommodation fields
+ 0.30 × (number of retrieved attraction names found in activity descriptions
        ÷ total number of retrieved attractions)
Final score clamped to [0.0, 1.0]
```

Example: 5 attractions retrieved, 3 mentioned in activities → $0.50 + 0.20 + 0.30 \times (3/5) = 0.88$

Metric 2 — Answer Relevance

Question: Does the generated plan actually address the user's stated interests?

Formula:

```
interests = user's stated interests (e.g. ['beach', 'nightlife', 'heritage'])
matched   = count of interests whose text appears anywhere in the itinerary JSON
score     = 0.55 + 0.45 × (matched / total_interests)
If user stated no interests → default score = 0.75
```

Example: 3 interests, 2 found in itinerary → $0.55 + 0.45 \times (2/3) = 0.85$

Metric 3 — Context Precision

Question: Did the data-gathering agents retrieve good quality context?

Formula:

```
score = 0.40 (base)
+ 0.20 if places_data contains at least one attraction
+ 0.20 if hotel_data contains at least one recommendation
+ 0.20 if weather_data contains forecast data
Maximum possible = 1.00
```

This metric rewards the system for successfully fetching all three types of contextual data. If any agent failed to retrieve data, this score drops.

Metric 4 — Context Recall

Question: Were all the days the user requested actually covered in the itinerary?

Formula:

```
score = number_of_days_generated / number_of_days_requested
Clamped to [0.0, 1.0]

Example: user asked 5 days, itinerary has 5 days → score = 1.0
Example: user asked 7 days, itinerary has 5 days → score = 0.71
```

Overall RAGAS Score

```
overall_ragas_score = (faithfulness + answer_relevance
                        + context_precision + context_recall) / 4
```

Metric	Data Used	Score Range
Faithfulness	itinerary, places_data, hotel_data	0.50 – 1.00
Answer Relevance	trip_preferences.interests, itinerary	0.55 – 1.00
Context Precision	places_data, hotel_data, weather_data	0.40 – 1.00
Context Recall	trip_preferences.days, itinerary.days	0.00 – 1.00

8. DeepEval Evaluation — 5 Metrics

File: `utils/evaluator.py` | Class: `DeepEvalEvaluator`

DeepEval measures the quality of the LLM's generated output — not how well it used retrieved data (that is RAGAS), but whether the output itself is coherent, accurate, safe, and unbiased.

Metric 1 — Coherence

Question: Is the itinerary structurally complete and well-formed?

Formula:

```
base          = 0.40
+ 0.20 if every day has both activities and a day number
+ 0.20 × (days with food field / total days)
+ 0.20 × (days with accommodation field / total days)
Maximum possible = 1.00
```

Example: 5 days, all complete, all have food, 4/5 have accommodation → $0.40 + 0.20 + 0.20 + 0.20 \times (4/5) = 0.96$

Metric 2 — Correctness

Question: Is the budget estimate within the user's stated budget limit?

Formula: Based on ratio = estimated_total / user_budget

```
ratio ≤ 1.00 → score = 0.97 (within budget – excellent)
ratio ≤ 1.05 → score = 0.85 (5% over – acceptable)
ratio ≤ 1.15 → score = 0.65 (15% over – warning)
ratio > 1.15 → score = 0.40 (significantly over – poor)
```

This is the most impactful metric. A plan that fits within budget gets near-perfect correctness. The budget agent applies a hard scaling cap so in practice this score is usually 0.97.

Metric 3 — Hallucination Score

Question: Does the plan contain suspiciously unrealistic values? **Lower is better** for this metric.

Formula:

```
suspicious = count of activities where cost > ₹50,000
total      = total number of activities across all days
score      = suspicious / total (lower = better, 0.05 if no activities)

Note: This score is INVERTED in the overall formula:
      overall uses (1 - hallucination_score)
```

A realistic Goa trip would never have a single activity costing ₹50,000+. If the LLM hallucinates such values, this metric catches it.

Metric 4 — Toxicity

Question: Does the generated plan contain harmful or illegal content?

Formula:

```
bad_words = ["illegal", "smuggle", "hack", "scam", "fraud", "terror"]
score = 0.10 if any bad word found in itinerary text
score = 0.98 if clean (higher = better)
```

■ This is a simple keyword check, not a semantic analysis. It can be bypassed with synonyms.

Metric 5 — Bias Score

Question: Does the plan show cultural or demographic bias?

```
score = 0.95 (hardcoded – not yet implemented)
```

■ This metric is a placeholder. Real bias detection would require LLM-based evaluation.

Overall DeepEval Score

```
overall = (coherence + correctness + (1 - hallucination)
          + toxicity + bias) / 5
```

Hallucination is inverted because 0 hallucination = perfect = 1.0 contribution

Metric	Data Used	Higher = Better?	Score Range
Coherence	itinerary structure	Yes	0.40 – 1.00

Correctness	budget_summary, trip_preferences	Yes	0.40 – 0.97
Hallucination	itinerary activity costs	No (inverted)	0.05 – 1.00
Toxicity	full itinerary text	Yes	0.10 or 0.98
Bias	hardcoded	Yes	0.95

9. Final Grade Calculation

```
overall_quality = (overall_ragas_score + overall_deepeval_score) / 2

Grade:
overall >= 0.85 → Excellent
overall >= 0.70 → Good
overall >= 0.55 → Fair
overall < 0.55 → Poor

passed_threshold = overall > 0.60
```

The grade and threshold are stored in `evaluation_results['summary']` and displayed in the Observability tab of the app as gauge charts.

10. Where Evaluation is Triggered

File: `agents/pdf_generator_agent.py` | Function: `pdf_generator_node`

Evaluation runs inside `pdf_generator_node`, which is the last agent before END. The order inside this node is:

```
1. OutputGuardrails().validate(state) # safety check on the output
2. run_evaluation(state)               # RAGAS + DeepEval scoring
3. TripPDFGenerator().generate(...)   # generate the actual PDF
```

Both guardrail results and evaluation results are saved into state under `guardrail_results` and `evaluation_results` respectively, regardless of whether PDF generation succeeds or fails.

11. Where Results are Displayed

File: `app.py` | Function: `render_results` | Lines: 983–1143

After the workflow completes, `render_results()` displays the evaluation scores in the **Observability tab** of the Streamlit UI as circular gauge charts.

UI Section	app.py Lines	Content
RAGAS Metrics card	1055–1077	4 gauges: Faithfulness, Answer Relevance, Context Precision, Context Recall + Overall De
DeepEval Metrics card	1079–1109	4 gauges: Coherence, Correctness, Toxicity, Bias + hallucination indicator + Overall De
Evaluation timestamp	1131–1139	Shows when evaluation ran

12. End-to-End Order of Execution

Below is the complete execution order from app startup to PDF download, showing exactly when memory and evaluation steps occur:

Step	Agent / Function	Memory / Eval?	What Happens
1	app.py → main()	—	User fills form, clicks Generate
2	InputGuardrails.validate_form_data()		Regex content check on form input
3	orchestrator_node	—	LLM decides: go to user_input
4	user_input_node	—	LLM parses raw text into structured trip_preferences JSON
5	orchestrator_node	—	LLM decides: go to memory_retrieval
6	memory_retrieval_node	READ	Fetches from FAISS + SQLite → user_preferences_memory
7	orchestrator_node	—	LLM decides: go to gather_data
8	weather_node	—	Fetches weather forecast
8	hotel_node	READ memory	Finds hotels — uses preferred_hotel_type from memory
8	transport_node	—	Finds transport options
8	places_node	—	Finds attractions and experiences
9	orchestrator_node	—	LLM decides: go to budget
10	budget_node	READ memory	Calculates budget — warns if below avg_budget
11	orchestrator_node	—	LLM decides: go to itinerary
12	itinerary_node	READ memory	LLM generates day plan — uses past destinations, travel style
13	orchestrator_node	—	LLM decides: go to review
14	review_node	—	Validates plan for conflicts, approves or requests retry
15	orchestrator_node	—	LLM decides: go to memory_update
16	memory_update_node	WRITE	Saves trip to FAISS vector store + SQLite both tables
17	orchestrator_node	—	LLM decides: go to pdf_generator
18	pdf_generator_node	—	OutputGuardrails check
18	run_evaluation(state)	EVAL	RAGAS (4 metrics) + DeepEval (5 metrics) scored
18	TripPDFGenerator.generate()	—	PDF written to output/
19	orchestrator_node → END	—	Workflow complete
20	app.py render_results()	—	Displays plan + gauge charts for RAGAS and DeepEval